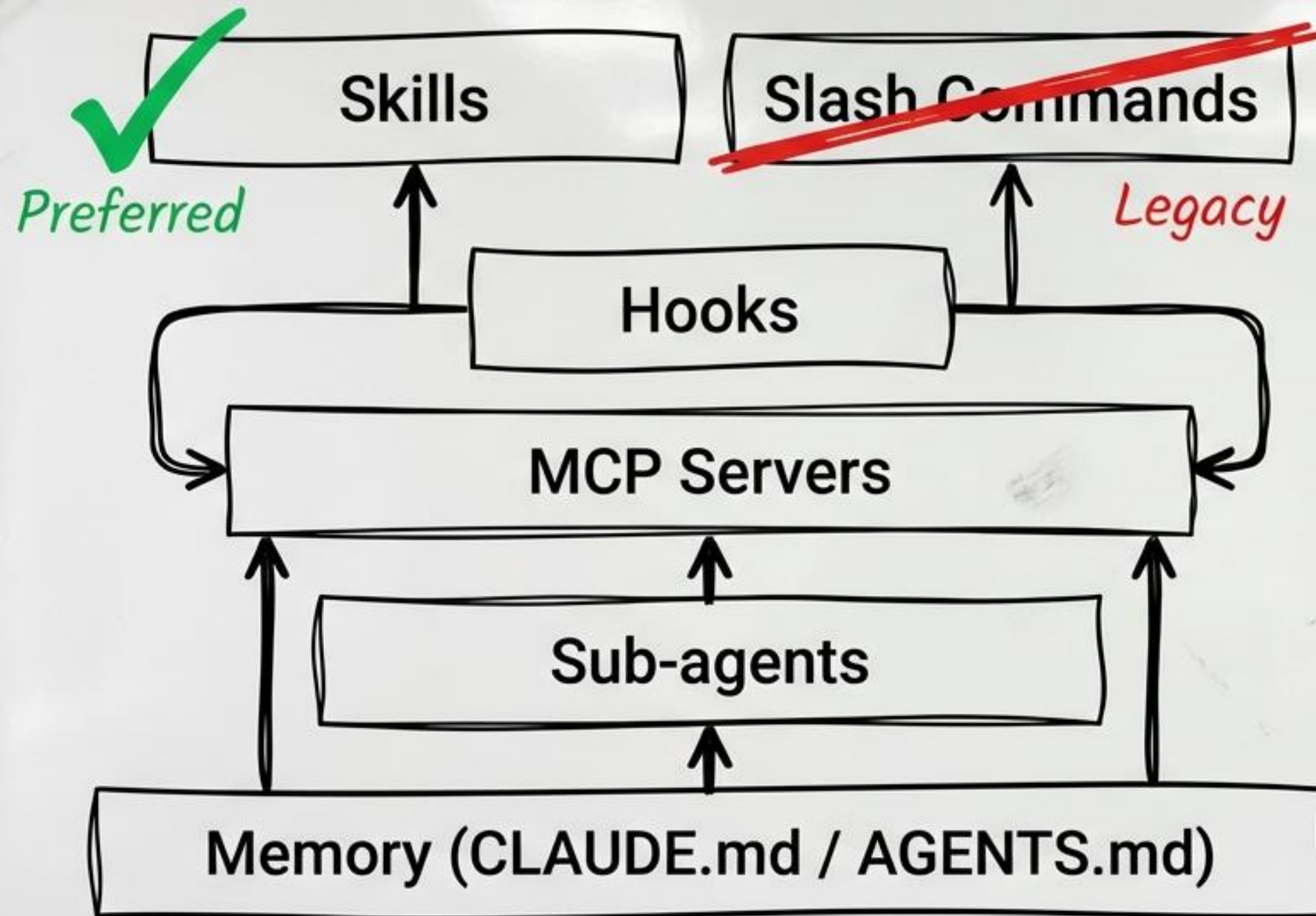


# Architecting Claude Code Environments

A pragmatist's guide to directory structures, token economics, and custom agent workflows.

Reminder:  
Token budget  
= my bonus.  
Keep it efficient.

# The Six-Component Customization Architecture



Unlike IDE-based tools, Claude Code is terminal-native and file-first.

Every artifact is a plain text file tracked in version control.



# The Complete Architectural Blueprint

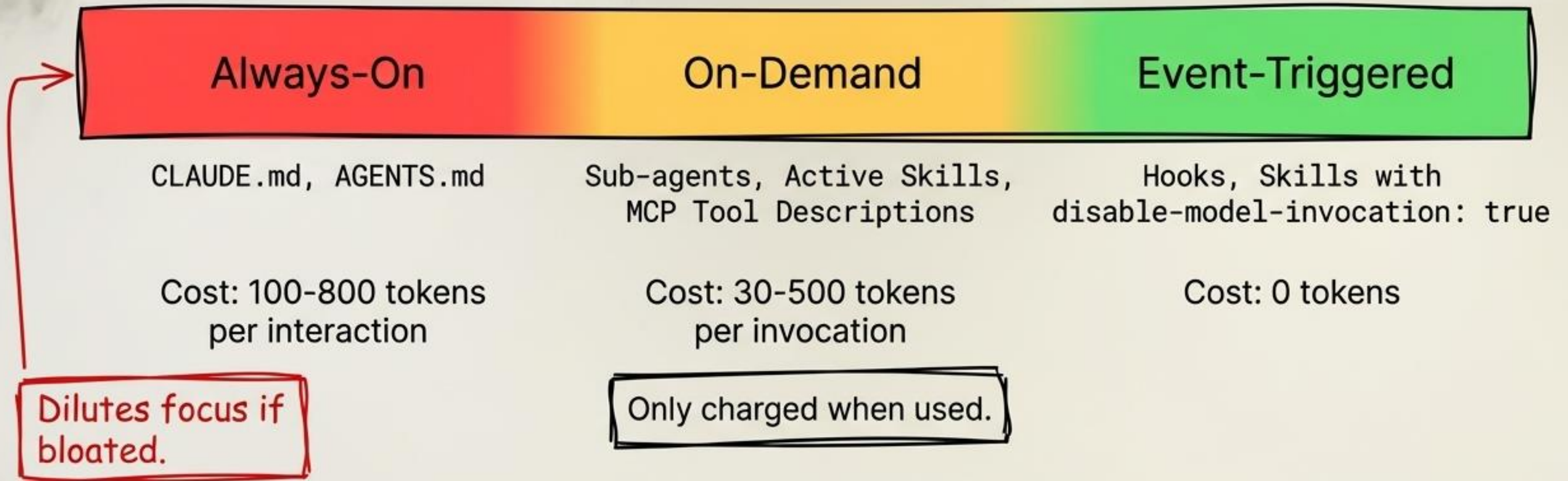
/ (Project Root)

- └─ CLAUDE.md / AGENTS.md ----- Always-on project memory.
  - └─ .claude/
    - └─ CLAUDE.md ----- **Caveat: Local overrides/secrets. DO NOT COMMIT.**
    - └─ settings.json ----- Hooks, MCP, Permissions.
    - └─ agents/\*.md ----- Project-scoped specialists.
    - └─ skills/<name>/SKILL.md -- **Modern auto-invoked workflows.**
  - └─ src/
    - └─ CLAUDE.md --- Module-scoped memory.
- ~/.claude/ --- User-global configs live here.

Add  
.claude/CLAUDE.md  
to .gitignore before  
security yells at us.



# The Token Cost Spectrum

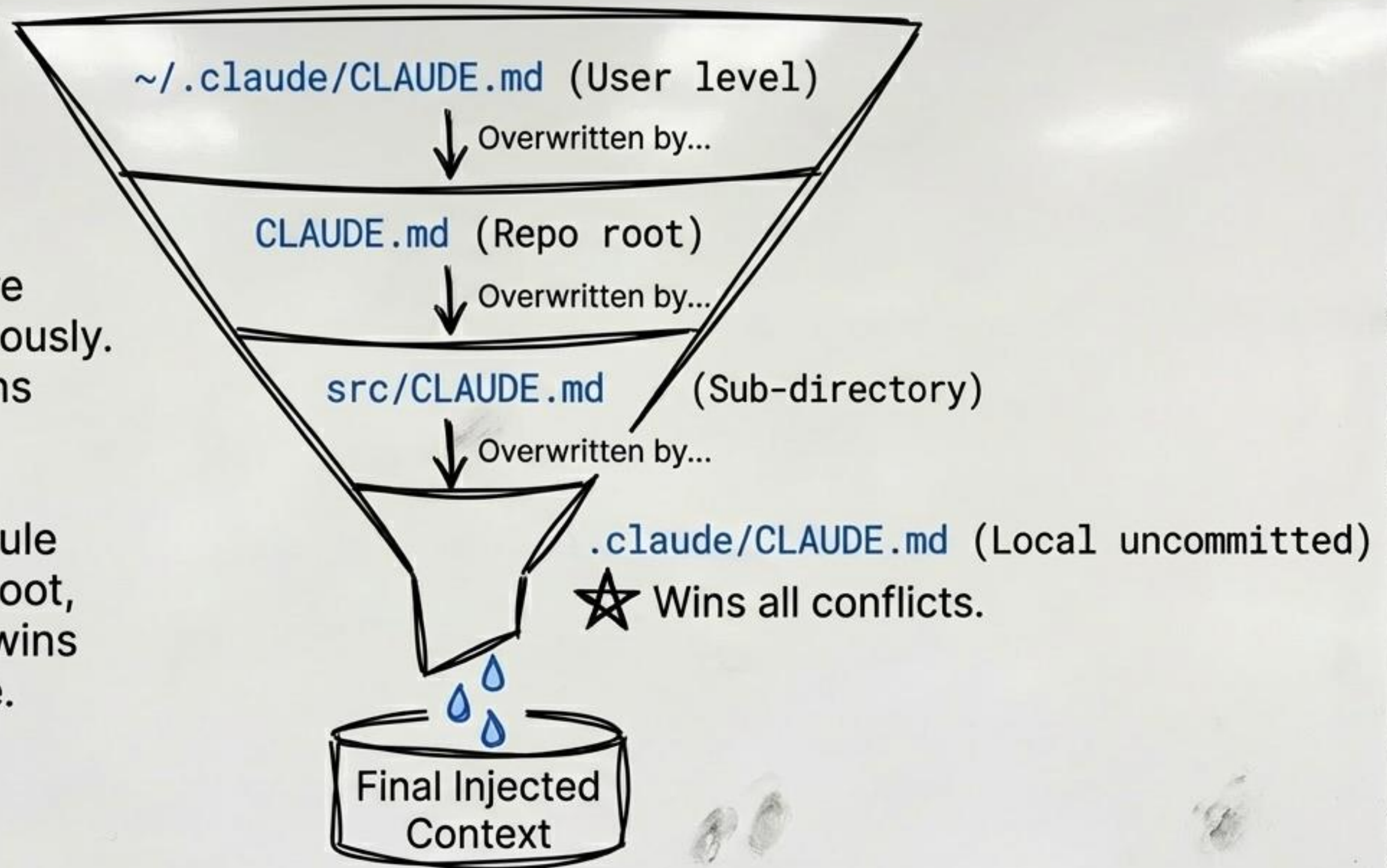


**Rule #1: Never always-on what can be on-demand.**



# Memory Composition and Precedence

- All in-scope files are provided simultaneously.
- Avoid contradictions between layers.
- If a sub-directory rule conflicts with the root, the sub-directory wins for files in that tree.





# AGENTS.md vs CLAUDE.md

## AGENTS.md

The Open Standard

- Code style conventions
- Naming conventions
- Architecture decisions
- Universal build commands

Reads by Claude, Copilot, Cursor, Jules.

*Put the "how we write code" stuff in AGENTS.md.  
Put the "how Claude behaves" stuff in CLAUDE.md.*

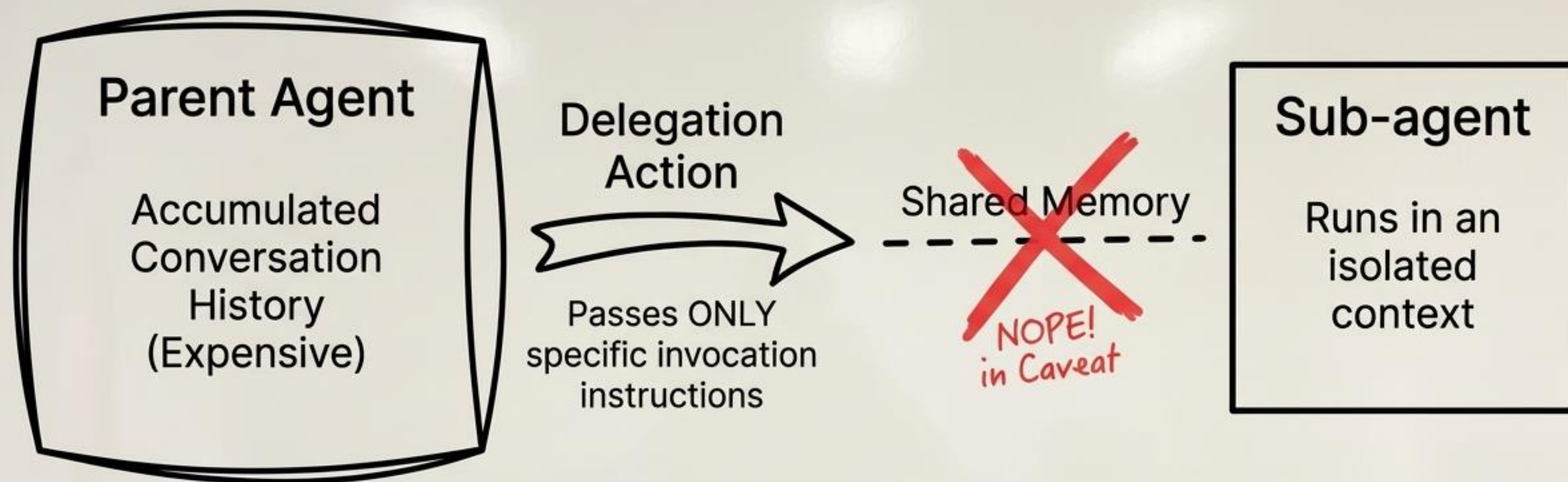
## CLAUDE.md

The Claude-Specific Engine

- Sub-agent workflows
- Claude-specific hooks docs
- Personal editor preferences
- Local uncommitted secrets



# Sub-agents: The Power of Context Isolation

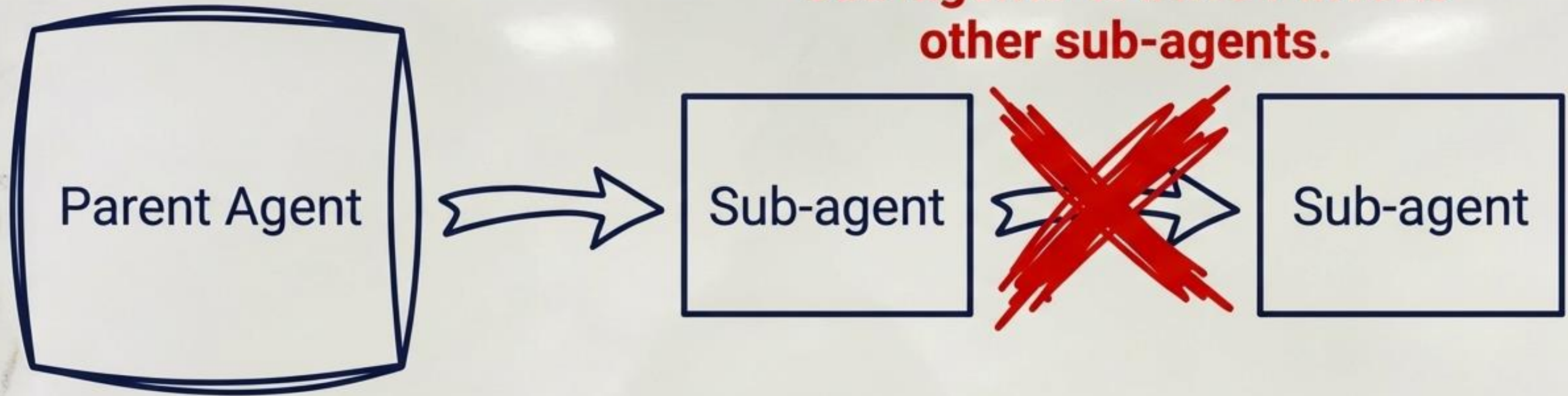


- ✓ **Security:** Cannot exfiltrate context it was never given.
- ✓ **Focus:** Entire context window dedicated to the delegated task.
- ✓ **Token Efficiency:** Parent history does not inflate the sub-agent cost.
- ✓ **Parallelism:** Shared-nothing architecture enables concurrency.



# The Strict Nesting Constraint

Sub-agents **CANNOT** invoke other sub-agents.

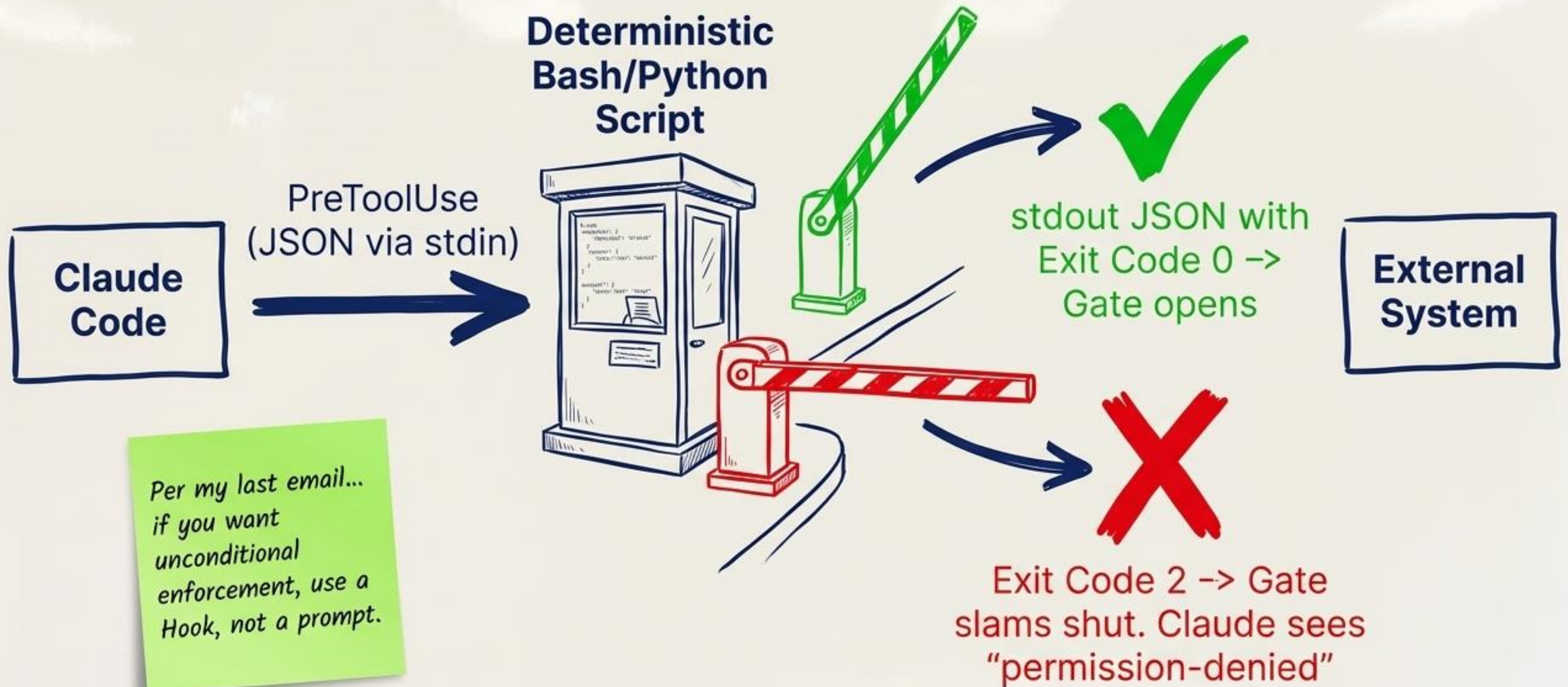


- The delegation hierarchy is single-level only.
- Prevents infinite recursion.
- Prevents privilege escalation chains.
- Prevents unbounded token consumption loops.

*Top-level parent must orchestrate.  
Here be dragons (and infinite billing loops).*



# Hooks: The Deterministic Security Tollbooth





# The “Zero Token” Magic Trick

**Heavy  
Prose Workflows  
in CLAUDE.md**

*“Always run the  
linter after editing a  
python file. Check for  
trailing commas...”*  
Caveat

150 Tokens  
Every Turn



**Deterministic  
Bash Hook**

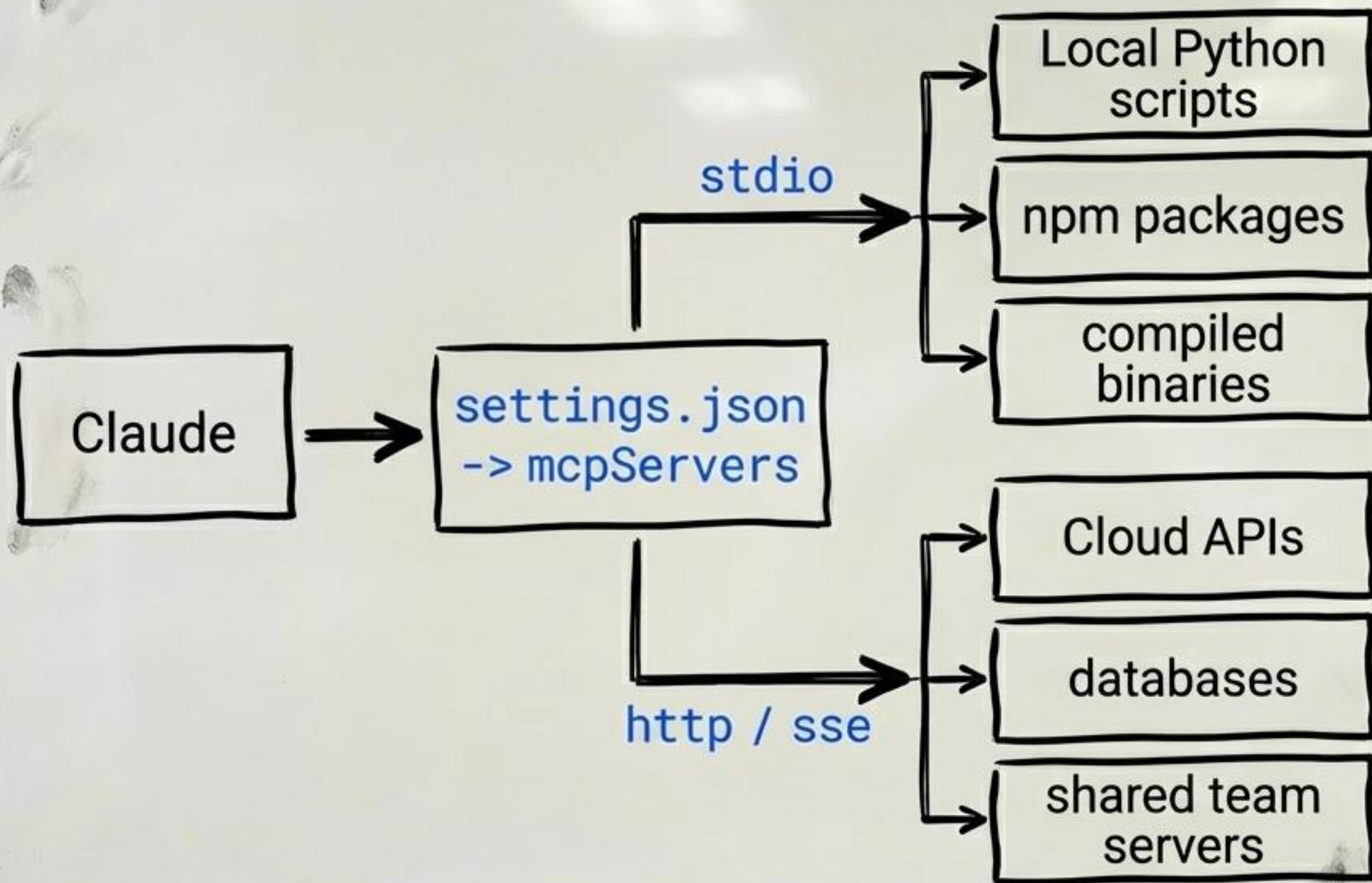
(PostToolUse trigger  
running 'black .')

0 Tokens.  
100% Reliable

Hooks execute at the process level and bypass the model entirely. They are the most cost-effective mechanism for enforcement, logging, and automation.



# Extending Capabilities with MCP Servers



## Security Warnings

- NEVER hard-code credentials in settings.json. Use `${ENV_VAR}` references.
- Audit third-party tool descriptions (prompt injection risk).
- Use `disallowedTools` blocklists in sub-agent frontmatter to restrict MCP access.



# Upgrading Workflows: Slash Commands vs. Skills

| Slash Commands (Legacy)                                                                                                           | Skills (Preferred)                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Storage:<br><u>.claude/commands/*.md</u>                                                                                          | Storage:<br><u>.claude/skills/&lt;name&gt;/SKILL.md</u>                                                                   |
| <u>Claude Auto-Invocation:</u>  <u>No</u>      |  <u>Yes (via description matching)</u> |
| <u>Isolated Subagent Context:</u>  <u>No</u> |  <u>Yes (context: fork)</u>          |
| <u>Cross-platform Standard:</u>  <u>No</u>   |  <u>Yes (Agent Skills spec)</u>      |

Skills supersede slash commands. They support auto-invocation, parallel execution, and open standards.



# Anatomy of a Skill Workflow

The description field isn't documentation. It's the literal trigger mechanism.

**SKILL.md**

**description:** "..."

Critical. Claude reads this to decide when to auto-trigger.

**disable-model-invocation:** true

Forces manual-only trigger. Saves tokens for heavy workflows.

**context:** fork

Spawns an isolated subagent. Prevents main-thread context bloat.

**\$ARGUMENTS[N]**

Dynamic input substitution at runtime.



# The Ultimate Anti-Pattern: The God File

## CLAUDE.md

coding\_standards.py: function names should be...  
... (prociousl... t, lano, autton, functions, lilen)  
debugging\_runbook\_v3.sh: if error 502, check...  
debugging\_runbook\_v3.sh: if error 502, check...  
ci\_scripts/build.yml: steps for deployment...  
debugging\_wakert... function names should ...  
debugging\_runbook\_v3.sh: if error 502, check...  
ci\_scripts/build.yml: steps for deployment...  
ci\_scripts/build.yml: steps for deployment...  
ci\_scripts/build.yml: steps for deployment...  
... scosegiarating\_ront... steps fo deployment, debugg... anbook\_v3.sh...  
editor\_hab...txt: use vim binding...  
editor\_hab...txt: use vim bindings...  
ci\_scripts/build.yml: steps for deployment...  
... pro... nswidenvillo.txt, debuggions-row-venement, isterhien, simple ...  
coding\_standards.py: fimm poom /Revviffr...

Engineers instinctively dump everything into **CLAUDE.md**. This is a massive failure of **separation-of-concerns**.

Why it fails:

1. **Token Waste**: Pay for the debugging workflow on every simple question.
2. **Dilution**: Procedural runbooks drown out the core coding standards.
3. **Unreliable**: LLMs struggle with highly procedural, conditional instructions.

Stop putting 15-step CI debugging guides in the always-on memory!



# Diagnostic Matrix: Where Does It Belong?

| The Requirement                        | The Tool                         |
|----------------------------------------|----------------------------------|
| Req: Always-on Global Rules →          | CLAUDE.md / AGENTS.md            |
| Req: Unconditional Enforcement →       | Hook (PostToolUse)               |
| Req: Complex, Auto-detected Workflow → | Skill ( <b>Auto-invoke</b> )     |
| Req: Heavy, User-triggered Task →      | Skill (disable-model-invocation) |
| Req: Automated Parallel Tasks →        | Sub-agent (.claude/agents/)      |
| Req: External DB/API Access →          | MCP Server                       |

✓ If it requires enforcement, use a Hook. If it's a specific task, use a Skill.



# Blast Radius Containment & Security

## Security Tripwires

### 1. Default Deny Shell Access

Set Bash(\*) to deny at the bottom of settings.json. Explicitly enumerate allowed commands (Bash(git:\*)).

```
// settings.json
"Bash(*)": "deny",
"Bash(git:*)": "allow",
...
```

 **BLOCKED**

### 2. Least Privilege Agents

Use disallowedTools blocklists in sub-agent frontmatter to restrict access.

```
// .claude/agents/sub-agent.md
---
disallowedTools: ["read_file", "write_file"]
---
```

### 3. Container Isolation

Run risky sub-agents (tests, migrations) inside Docker containers.

```
docker run --rm -it my-test-agent
```



### 4. The Red Flag

Explicitly audit any sub-agent using permissionMode: bypassPermissions.

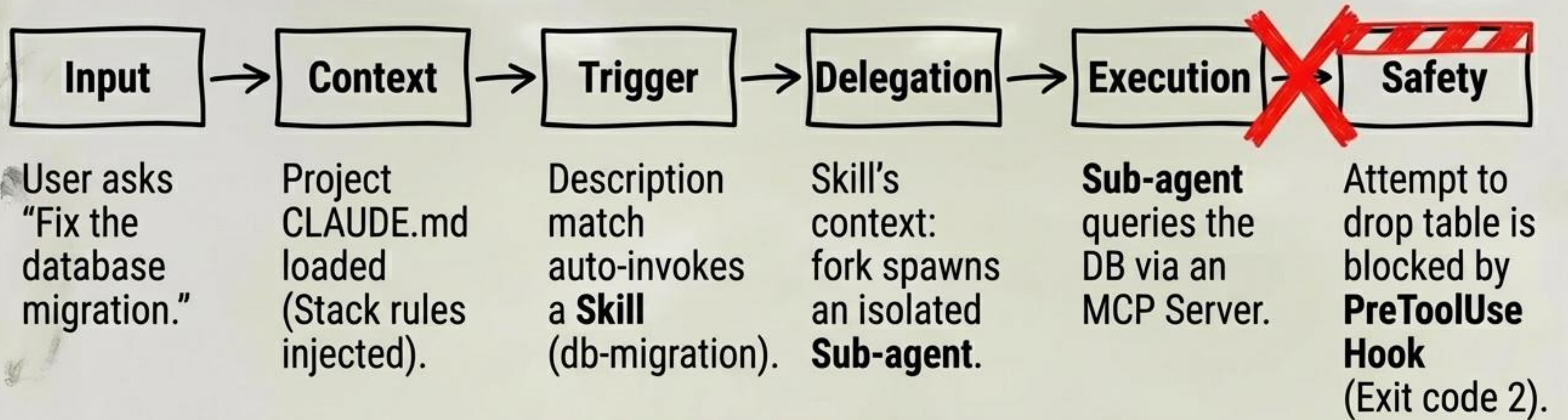
```
// .claude/agents/risky.md
permissionMode: "bypassPermissions"
```

**This is a HUGE security risk! Review ASAP.**





# The Unified Request Lifecycle



*A beautifully orchestrated system where no single file does all the work.*



# Architecture Comparison: Claude Code vs. Copilot

| Feature                 | Claude Code                | GitHub Copilot          |
|-------------------------|----------------------------|-------------------------|
| Environment             | Terminal-first             | IDE-first               |
| Cross-platform standard | Both support AGENTS.md     | Both support AGENTS.md  |
| Sub-agent Nesting       | Single-level restricted    | Single-level restricted |
| Hooks                   | Tool-level stdin/stdout    | Event-level             |
| Workflows               | Agent Skills open standard | Proprietary prompts     |

**Claude Code treats AI as a disciplined, version-controlled CLI tool. Every enforcement mechanism is a script, and every context decision is auditable.**



# The Terminal-Native Philosophy ✓

The goal is not just to make Claude helpful. It is **to make Claude reliably correct within the specific, auditable constraints of your project.**

- Every customization is a plain text file.
- Every enforcement mechanism is a script.
- Every context decision is observable.

*End of session.  
Wipe the board.*